



Quiz IA — Génération et scoring d'idées de quiz vidéo virales bilingues (FR/EN)

Rapport de projet de fin d'étude — LIORA

PROJET DE FIN D'ÉTUDES

DATA SCIENCE

YOUTUBE · RAG · LIGHTGBM

Loïc FONKOUA

[Code source du projet \(GitHub\)](#)

[Voir la chaîne YouTube](#)

1. Introduction

1.1 Contexte

La chaîne publie environ une vidéo par jour, en français et en anglais. La production est automatisée ; le goulot d'étranglement est désormais l'idéation créative quotidienne : trouver des idées qui sortent du lot et ont un potentiel viral. Les concurrents publient 1,5 à 2 vidéos par jour, l'enjeu concurrentiel est réel.

Sur le plan technique, le système combine deux briques : un générateur d'idées par récupération de contexte (RAG) et un modèle de scoring viral supervisé, entraîné sur des métadonnées de vidéos concurrentes collectées via l'API YouTube. L'ensemble est industrialisé avec collecte, pipeline reproductible, API, démonstration et tâche quotidienne.

Économiquement, le temps d'idéation constitue le coût marginal principal d'un créateur solo. L'automatiser me permet de tenir un rythme bilingue soutenu et de concentrer l'effort sur les idées à plus fort potentiel.

Scientifiquement, ce projet mobilise deux champs : la prédiction supervisée de popularité (régression sur données tabulaires) et la génération de texte contrainte et ancrée (RAG). Prédire la viralité *avant publication* est un problème ouvert et bruité — un cas d'étude représentatif des défis réels de la data science appliquée aux médias numériques.

1.2 Objectifs du projet



Collecter et structurer

Corpus de vidéos concurrentes étiqueté par famille × catégorie × langue.



Définir la cible

Variable de viralité robuste et comparable entre niches.



Entraîner un modèle

Prédire la viralité à partir de signaux disponibles avant publication.



Générer et classer

Idées ancrées (RAG) classées par potentiel viral prédit.



Industrialiser

API, démonstration, mise à jour quotidienne automatisée.

Expertise : Je réalise ce projet seul. Créateur de la chaîne concernée, j'ai une bonne connaissance des formats, des niches et de l'audience, ce qui a guidé mes choix tout au long de la formation.

1.3 Architecture du système

Le système s'organise en sept couches, de la collecte des métadonnées jusqu'à leur consommation par l'application de production. Le flux se lit de gauche à droite sur la rangée du haut, puis de droite à gauche sur celle du bas. La flèche en pointillés referme le circuit : chaque matin, la chaîne est relancée depuis la collecte, sans aucune intervention de ma part.

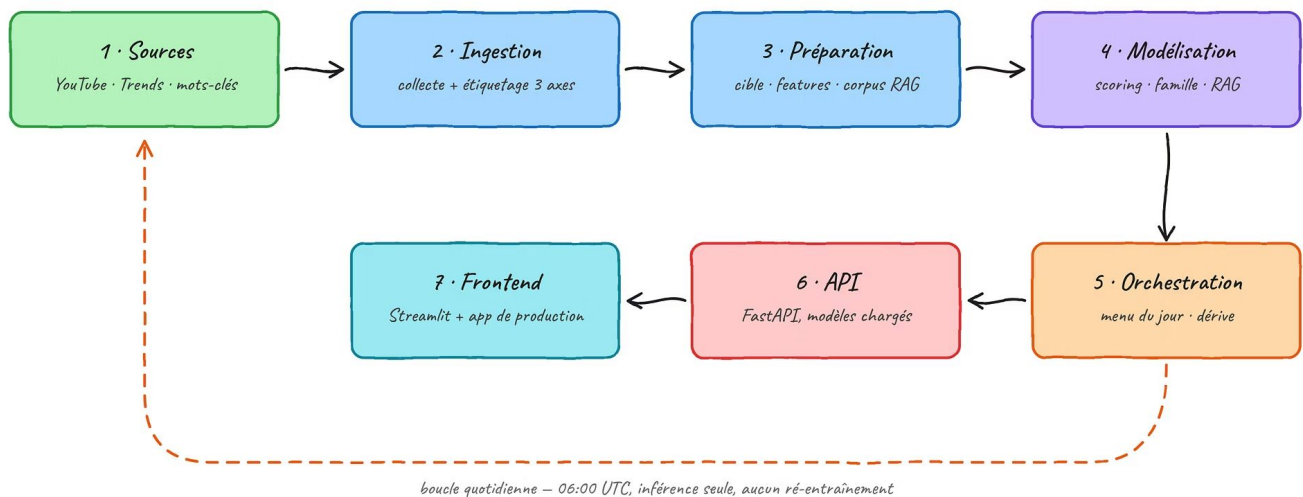


Schéma 1 — Architecture du système, de la collecte des données à l'intégration de l'API dans l'application de production.

2. Compréhension et manipulation des données

2.1 Cadre de collecte

Le jeu de données repose sur les métadonnées de vidéos YouTube concurrentes, collectées via la **YouTube Data API v3** (source légale et officielle), complétées par des sujets tendance issus de Google Trends et des mots-clés agrégés. Les métadonnées sont entièrement publiques et ne contiennent aucune donnée personnelle.

476

Vidéos collectées

7 catégories × 2 langues

350

Mots-clés

Agrégés par niche

171

Sujets tendance

Google Trends

2.2 Construction de la variable cible

La viralité brute (vues) n'est pas comparable d'une vidéo à l'autre, car elle dépend de l'ancienneté de la vidéo et de la taille de la niche. On construit donc un score relatif en trois étapes :

```
views_per_day = vues / ancienneté (jours)
virality_log = log(1 + views_per_day)
virality_score = z-score de virality_log
                au sein de chaque (catégorie × langue)
```

Le **virality_score** mesure la performance relative à la niche et à la langue : 0 correspond à une vidéo moyenne, une valeur positive indique une **surperformance**. C'est la cible du modèle supervisé.

2.3 Particularités et limites du corpus

Déséquilibre linguistique

EN = 290 vidéos (61 %), FR = 186 (39 %).
Sur-représentation de l'anglais — biais de collecte à garder en tête pour l'interprétation des résultats.

Format long dominant

Durée médiane $\approx$ 13 min. Le corpus est majoritairement composé de vidéos en format long, ce qui impacte les features de durée.

Familles manquantes

Les vidéos ne sont pas pré-étiquetées : la famille est déduite par règles, avec 40,1 % de valeurs manquantes — traitées par un classifieur dédié.

Outliers extrêmes

Médiane à 442 vues/jour, maximum à $\sim$ 95 660. La transformation log + z-score gère ces extrêmes sans les supprimer.

- ❏ Les trois axes d'étiquetage — langue (FR/EN), catégorie (7 valeurs), et famille (inférée) — structurent toute la modélisation et la normalisation de la cible.

2.4 Dictionnaire de données principal — videos.csv (476 lignes)

Colonne	Type	Description	Dispo. a priori	Taux NA	Gestion NA
video_id	texte	Identifiant YouTube unique	—	0 %	Clé, non nulle
title	texte	Titre de la vidéo	oui	0 %	—
published_at	date	Date de publication (ISO 8601)	oui	0 %	Sert au calcul d'ancienneté
duration_iso	texte	Durée ISO 8601	oui	0 %	Parsée en secondes
view_count	entier	Nombre de vues	non	0 %	Base de la cible, exclue des features
like_count	entier	Likes	non	1,7 %	Exclue (fuite de données)
language	catégoriel	Axe 1 — FR / EN détectée	oui	0 %	Repli sur la langue de collecte
category	catégoriel	Axe 2 — chaîne exploitée (7 valeurs)	oui	0 %	Connue par la requête de collecte
family	catégoriel	Axe 3 — famille de quiz (inférée)	oui	40,1 %	Complétée par le classifieur (§5.2)

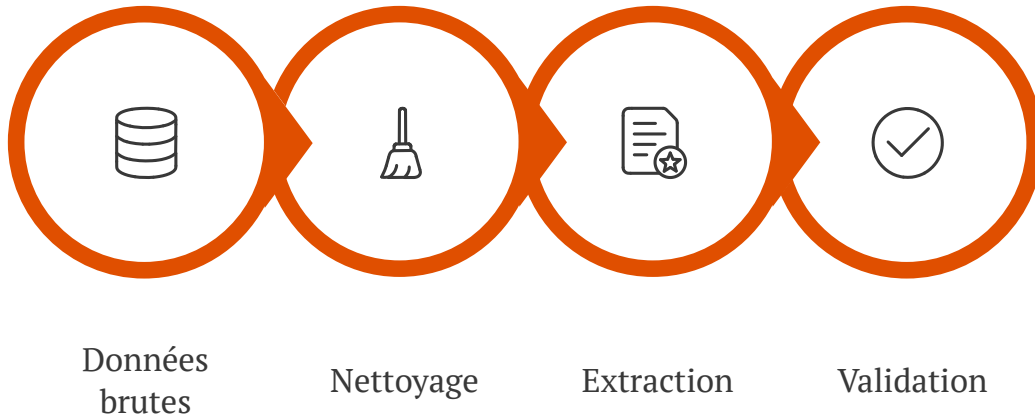
3. Pre-processing et feature engineering

Nettoyage des données

La première étape consiste à assurer la qualité et la cohérence du corpus brut. Le dédoublement est réalisé par `video_id`, garantissant l'unicité de chaque entrée. Les dates sont parsées au format ISO 8601 et les durées converties en secondes pour servir de feature numérique directement exploitable. La détection de langue vient compléter les déclarations manquantes, et l'étiquetage sur trois axes — catégorie, langue, famille — est appliqué systématiquement.

3.2 Feature engineering

Les features extraites couvrent plusieurs dimensions : les features de titre (longueur en caractères et en mots, présence d'un chiffre, d'un emoji, d'une marque d'interrogation ou d'exclamation, ratio de majuscules), la durée en secondes, le jour et l'heure de publication, et l'encodage one-hot des axes catégoriels (catégorie, langue, famille). L'espace de features résultant comporte environ 30 colonnes.



3.3 Principes directeurs du pipeline



Anti-fuite

Les vues, likes et commentaires sont strictement exclus des features. Un garde-fou bloque toute colonne interdite.



Interprétabilité

Pas de PCA : l'espace (~30 colonnes) est modeste. La réduction nuirait à l'interprétation SHAP.



Reproductibilité

Pipeline structuré en modules Python versionnés, exécutable de bout en bout sans intervention manuelle.

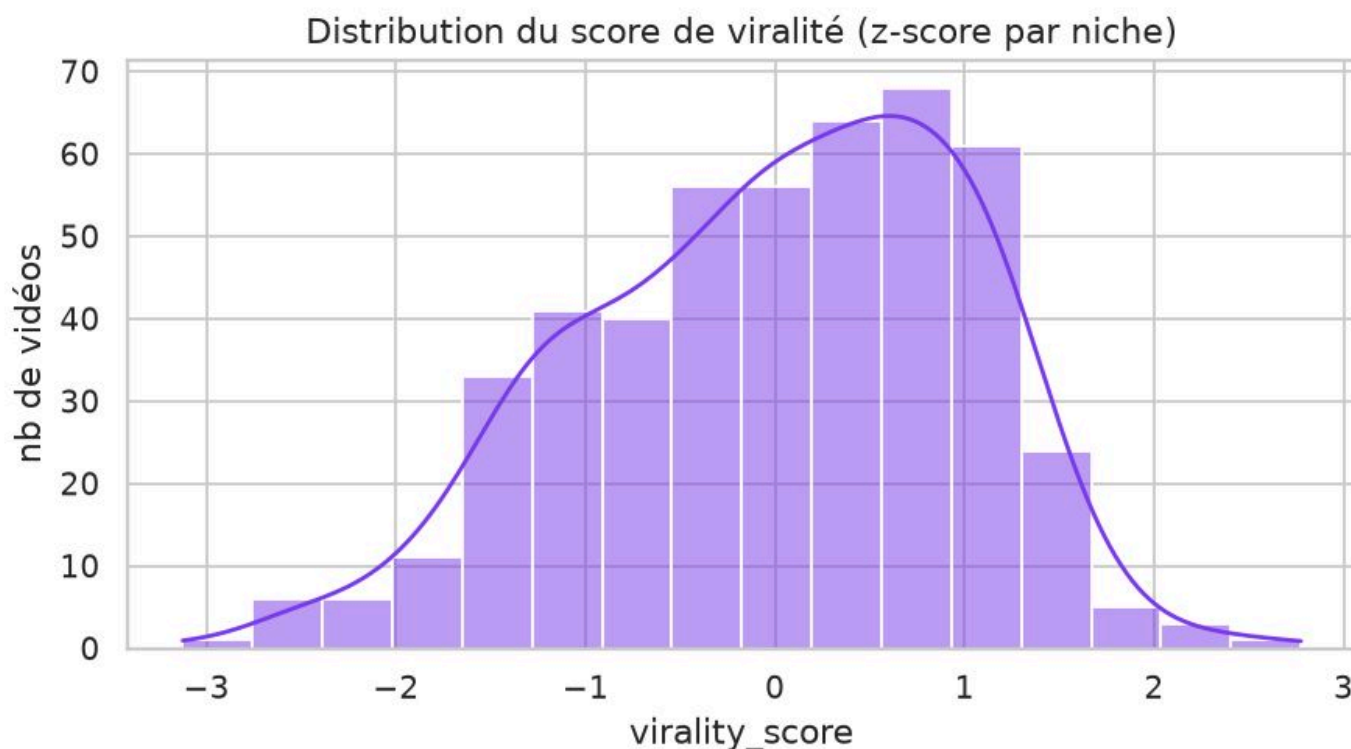


Règle anti-fuite : Le modèle n'apprend que sur des signaux disponibles avant publication. La cible étant dérivée des vues, toute métrique de performance post-publication est bannie des features d'entraînement.

4. Visualisations et statistiques exploratoires

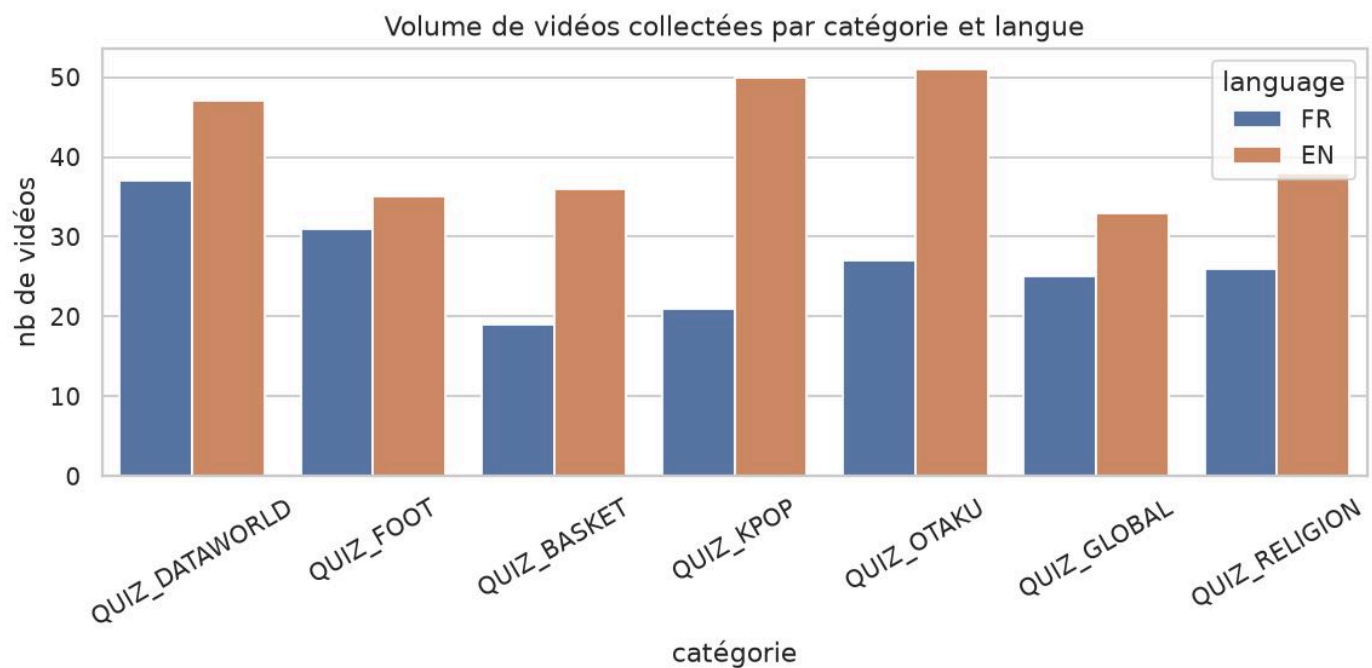
Chaque figure produite est commentée et validée par un test statistique approprié. L'exploration vise à justifier les choix de modélisation, notamment la normalisation de la cible et la sélection des features.

Fig. 1 — Distribution du virality_score



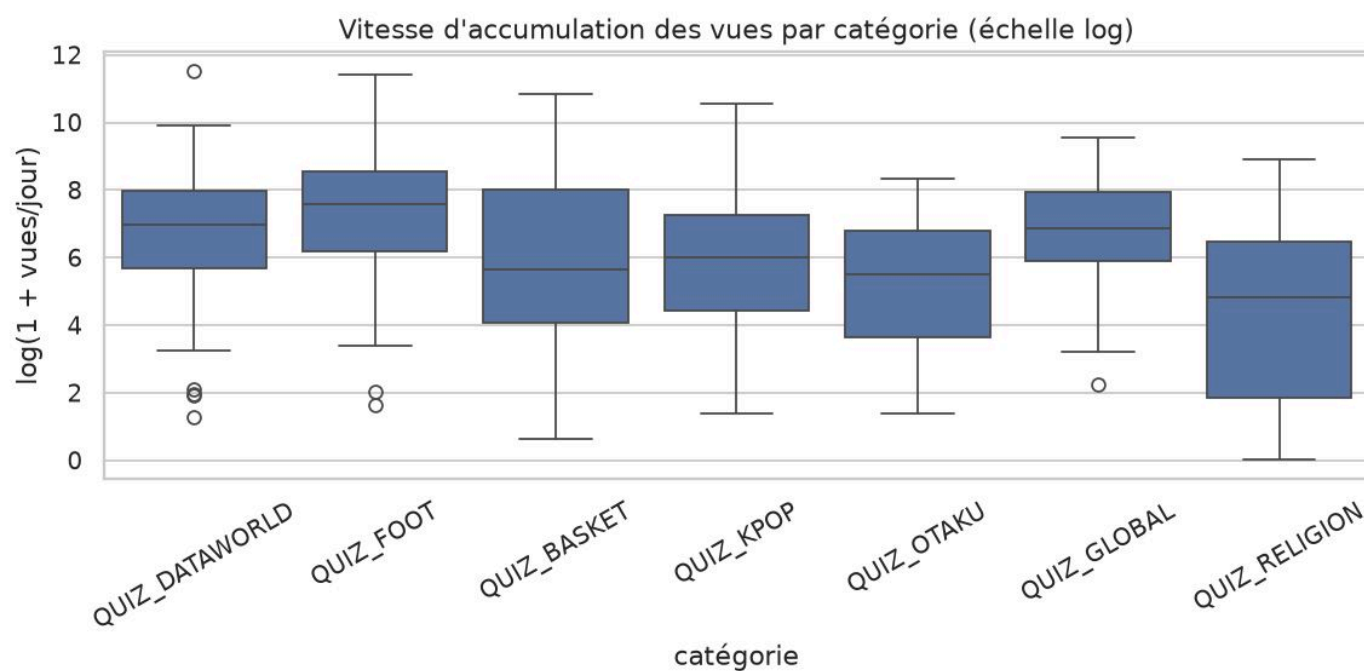
Après normalisation, les vidéos se répartissent de façon presque symétrique autour d'une vidéo moyenne. Une minorité nette surperforme franchement sa niche : ce sont les modèles à imiter. Les cas extrêmes sont conservés et non supprimés — la transformation suffit à les maîtriser.

Fig. 2 — Volume par catégorie et langue



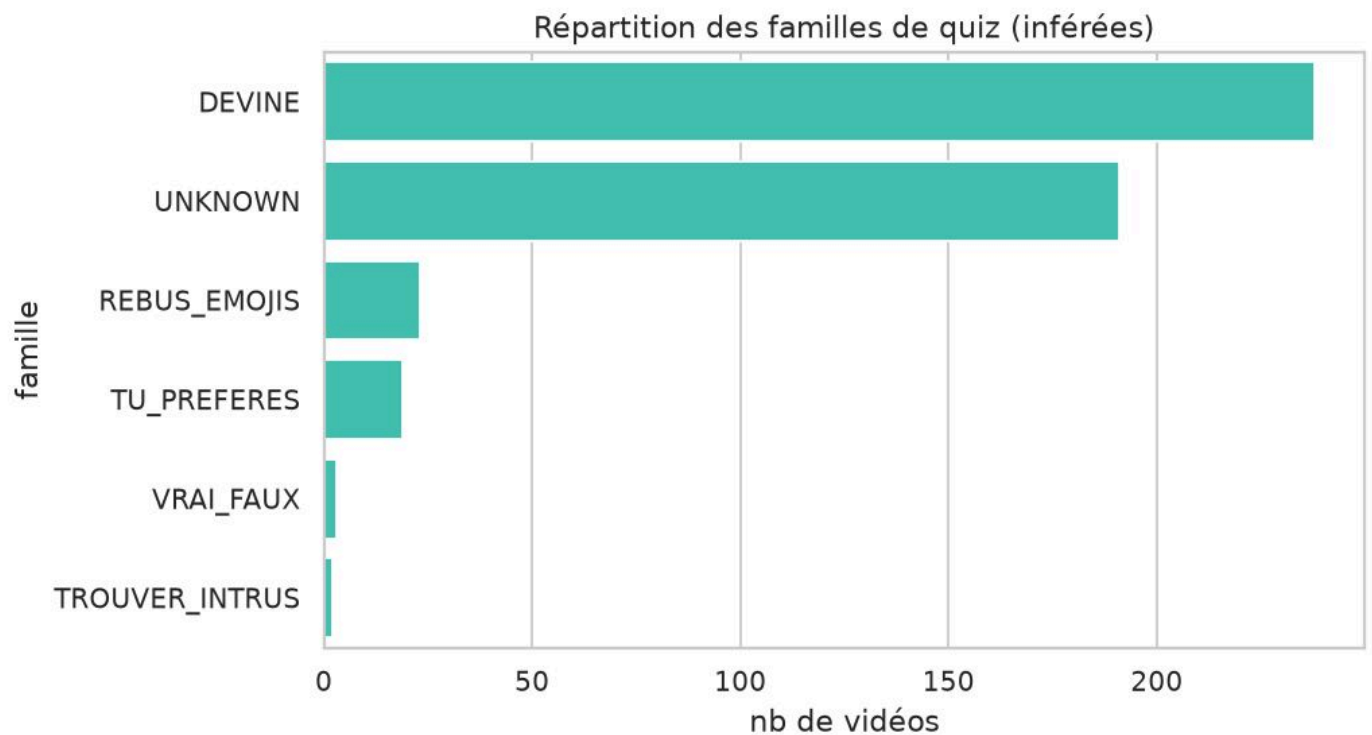
L'anglais domine largement le corpus, près de six vidéos sur dix. L'écart est trop marqué pour relever du hasard. C'est un biais de collecte que j'assume et que je garde en tête pour interpréter tout résultat bilingue.

Fig. 3 — Vues/jour par catégorie



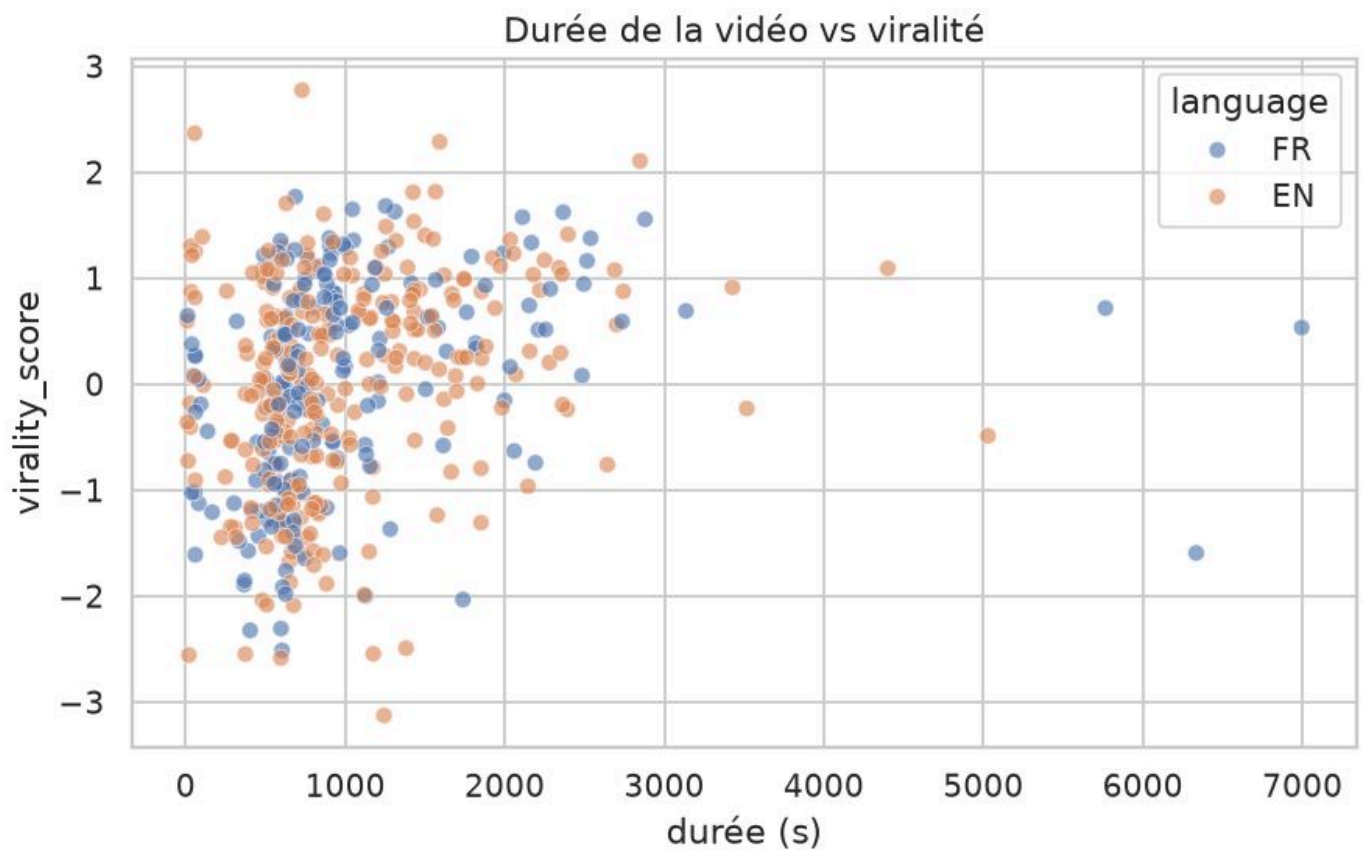
La vitesse d'audience varie fortement d'une niche à l'autre. Une bonne vidéo dans une petite niche fait moins de vues qu'une vidéo banale dans une grosse niche. C'est ce qui justifie de normaliser la cible par niche au lieu de comparer les vues brutes.

Fig. 4 — Répartition des familles



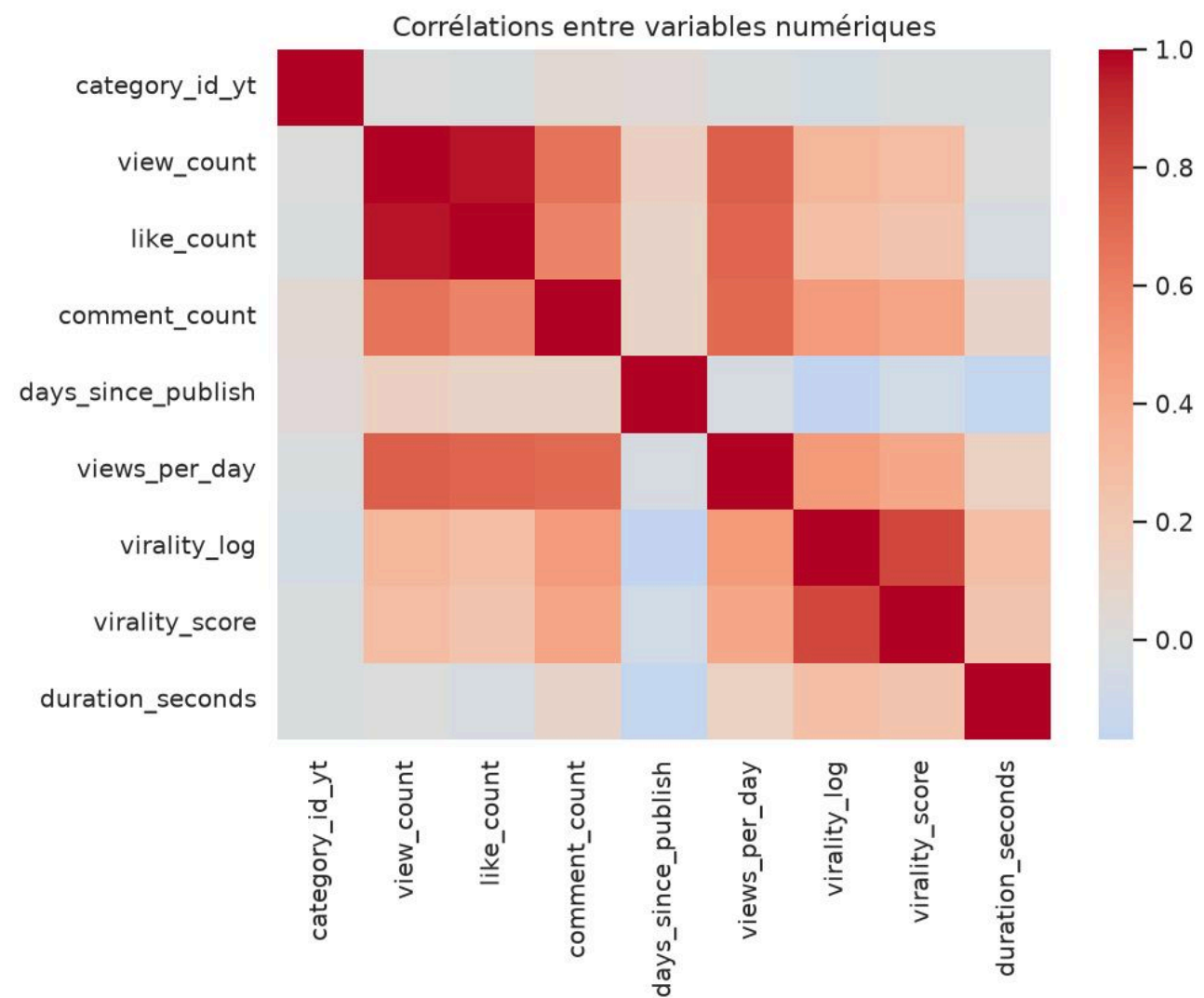
Une famille écrase toutes les autres, et quatre vidéos sur dix ne sont reconnues par aucune règle. Ce double problème m'a conduit à construire un classifieur de famille dédié (§5.2).

Fig. 5 — Durée vs viralité



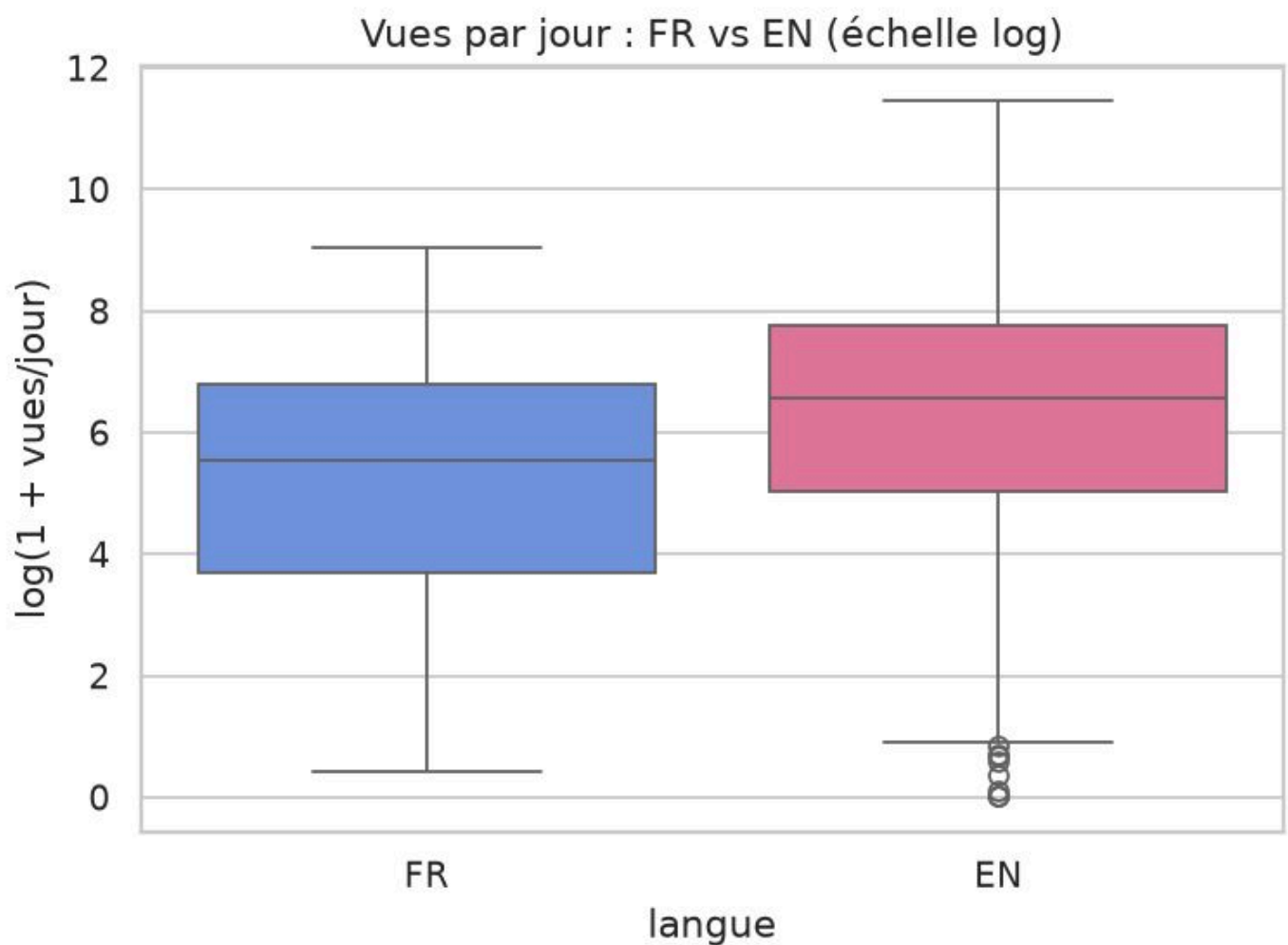
Les vidéos longues tendent à mieux performer, mais le lien reste modéré et probablement trompeur : les grosses chaînes font des vidéos longues et accumulent des vues pour d'autres raisons. Le simple étiquetage court/long, lui, ne distingue rien.

Fig. 6 — Matrice de corrélations des features



Les variables sont globalement indépendantes les unes des autres. Seules les deux mesures de longueur du titre se recouvrent, ce qui est logique puisqu'elles mesurent la même chose. Aucune réduction de dimension n'est nécessaire, ce qui préserve la lisibilité des explications du modèle.

Test complémentaire FR vs EN



Les vidéos anglophones accumulent nettement plus de vues par jour que les francophones — marché plus vaste, mais aussi plus concurrentiel. J'intègre ce signal comme variable du modèle.

- Conclusion de l'exploration : je retiens une cible à normaliser par niche (validé statistiquement), j'ai identifié la durée comme prédicteur fort mais à surveiller pour le confondant, et je documente le biais de langue et les familles manquantes comme limites principales. Je confirme la régression sur `virality_score` comme approche pertinente.

5. Modélisation

RÉGRESSION

CLASSIFICATION

RAG

SHAP

5.1 Scoring viral — Régression sur virality_score

La tâche est une **régression** : prédire le virality_score continu. Elle s'apparente à un problème de *ranking* — on cherche surtout à bien ordonner les idées par potentiel viral. La **corrélation de Spearman** est la métrique principale (qualité du classement), complétée par MAE et RMSE. L'objectif que j'ai fixé (H2) est Spearman > 0,5.

La démarche suit un protocole rigoureux : split train/test 80/20 avec validation croisée 5-fold. Quatre modèles sont comparés, de la baseline triviale au gradient boosting optimisé.

Modèle	Spearman	MAE	RMSE
Baseline — DummyRegressor	0,000	0,866	1,025
Régression linéaire	0,361	0,798	0,951
LightGBM (retenu)	0,476	0,737	0,926
LightGBM optimisé (GridSearch)	0,428	0,765	0,932

LightGBM bat nettement les baselines. Fait notable : l'optimisation par GridSearch ne surpasse pas le modèle par défaut sur ce petit volume (variance élevée) — phénomène documenté et que j'assume méthodologiquement.

Choix des familles de modèles

Boosting
(LightGBM)

État de l'art sur données tabulaires de cette taille. Je le retiens comme modèle principal.

Bagging
(Random Forest)

J'ai écarté cette option : apport marginal face au boosting sur 476 lignes, sans gain attendu.

Deep Learning

Je n'entraîne pas de réseau sur le scoring : sur-dimensionné pour 476 exemples et ~30 features.

NLP / Neurones → RAG

J'ai choisi de porter le volet neuronal par le générateur RAG (vecteurs + LLM). Choix méthodologique conscient.

5.2 Classifieur de famille – Classification multiclass

RÉGRESSION

CLASSIFICATION

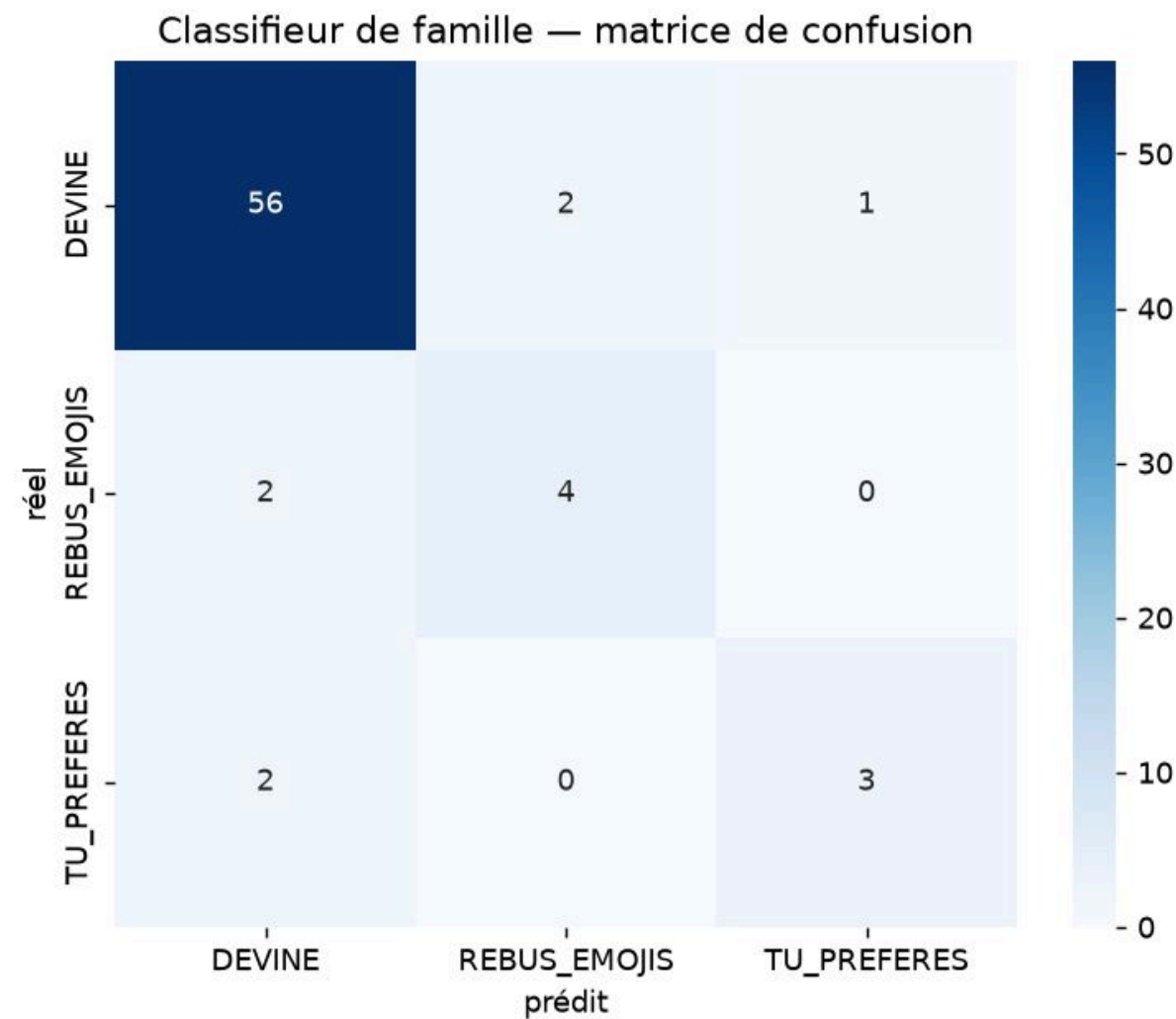
RAG

SHAP

Pour combler les 40 % de familles manquantes, la tâche est traitée comme une **classification multiclass** (titre → famille). La métrique adaptée au déséquilibre des classes est le macro-F1.

Modèle	Accuracy	Macro-F1
Baseline (classe majoritaire)	0,84	0,30
Régression logistique (équilibrée)	0,76	0,53
LightGBM (équilibré) [retenu]	0,90	0,76

Validation croisée macro-F1 : $0,69 \pm 0,17$. Le contraste baseline (0,84 d'accuracy mais 0,30 de macro-F1) illustre l'importance critique du choix de métrique en présence de déséquilibre de classes. J'ai ainsi comblé 191 familles manquantes.



5.3 Génération d'idées — Architecture RAG

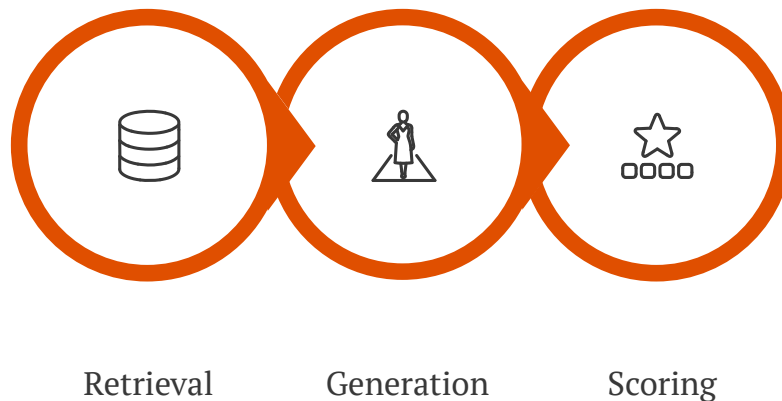
RÉGRESSION

CLASSIFICATION

RAG

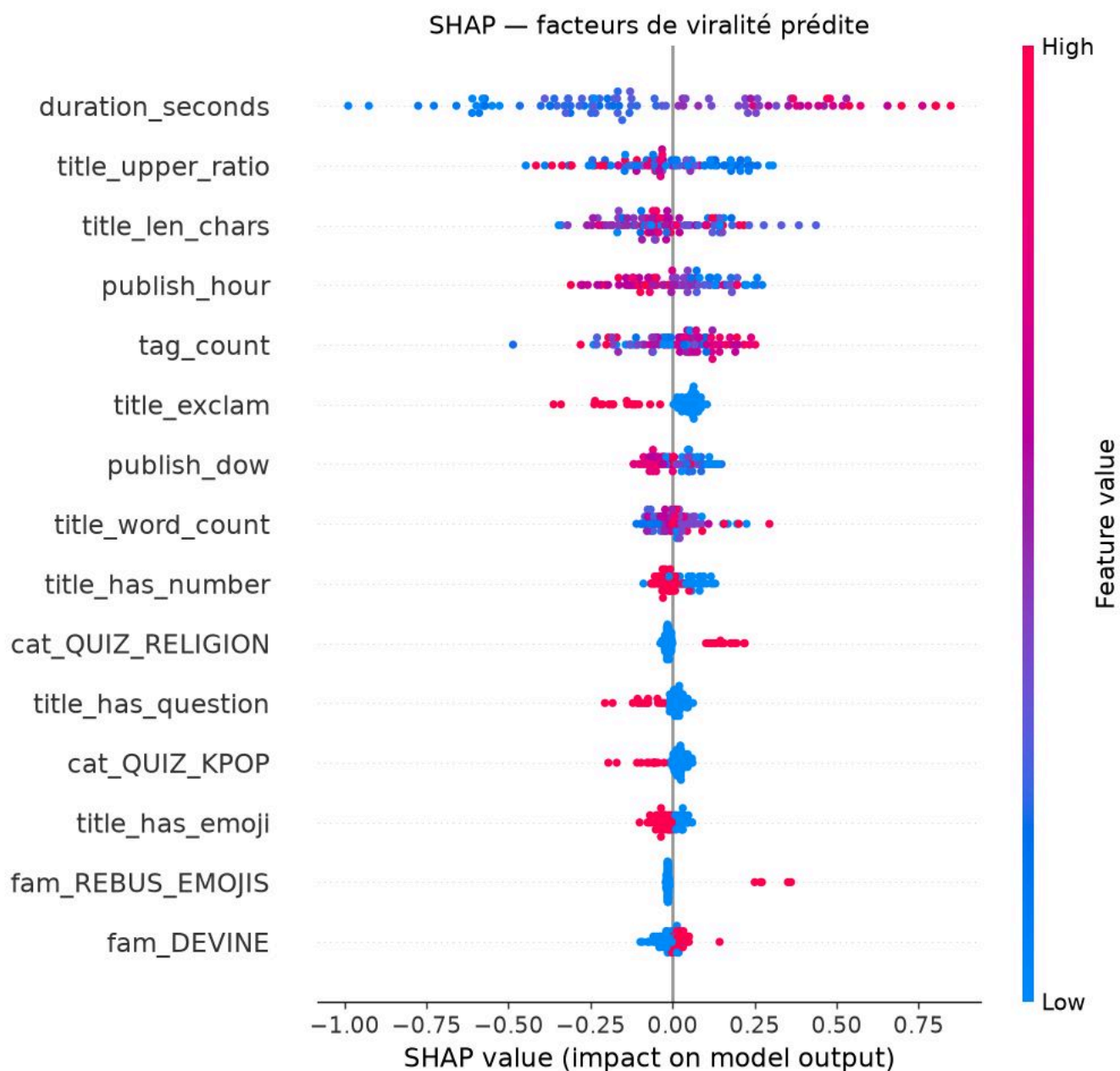
SHAP

L'architecture est 100 % RAG, sans fine-tuning. Pour une niche donnée, les titres performants, les tendances et les mots-clés sont récupérés et fournis en contexte à un LLM via API, qui produit des idées. Chaque idée est ensuite notée par le modèle de scoring et classée.



J'ai choisi TF-IDF pour le retrieval (corpus petit et spécialisé, hors-ligne, gratuit). J'ai validé le pipeline de bout en bout et la génération réelle fonctionne en production.

5.4 Interprétation SHAP — Top features du modèle de scoring



La durée pèse le plus lourd dans la prédiction, devant les caractéristiques du titre. Ces leviers sont directement actionnables par le créateur au moment de concevoir la vidéo. L'analyse SHAP (SHapley Additive exPlanations) permet d'identifier la contribution de chaque feature à la prédiction.

6. Conclusions

Ce projet aboutit à un système complet et opérationnel que j'ai construit de bout en bout : collecte étiquetée → features anti-fuite → scoring viral → classifieur de famille → génération RAG → API + démonstration → tâche quotidienne automatisée. L'architecture tient de bout en bout et se boucle seule.

H1 — RAG > zero-shot En cours Pipeline validé et génération réelle fonctionnelle. L'évaluation comparative formelle reste à formaliser avec un protocole d'annotation humaine.	H2 — Spearman > 0,5 Non atteinte Score obtenu : 0,476. Objectif frôlé mais non franchi. Les baselines sont néanmoins nettement battues — résultat attendu et cohérent pour un premier corpus de 476 vidéos.	H3 — Qualité ∝ retrieval À évaluer La relation entre la pertinence du retrieval et la qualité des idées générées reste à mesurer formellement dans les prochaines étapes du projet.
--	--	--

Conclusion nuancée et honnête : L'approche que j'ai retenue est pertinente — les modèles battent nettement les baselines, l'architecture tient et se boucle seule — mais la performance prédictive reste à consolider. Ce résultat est attendu et représentatif des défis réels de la prédiction de popularité sur un premier corpus de taille modeste, dans un domaine bruité par nature.

Retrouvez le code complet et commenté sur GitHub :

github.com/CerebroLeague/projet_data_Liora

Retrouvez les vidéos générées par le système sur la chaîne YouTube :

youtube.com/@CerebroLeagueFR

7. Difficultés et verrou scientifique

Verrou scientifique principal

Prédire la viralité **avant publication**, à partir de signaux de surface uniquement, sur un petit corpus bruité et avec des variables confondantes (notamment la durée), constitue le verrou central de ce projet. Ce problème est intrinsèquement difficile et partiellement irréductible : une part significative de la viralité relève de dynamiques sociales imprévisibles, d'effets de réseau et de contexte temporel impossible à capturer dans les métadonnées seules.

La littérature scientifique confirme que même les meilleurs systèmes atteignent des corrélations modérées sur ce type de tâche. L'objectif de Spearman $> 0,5$, que j'avais fixé comme seuil ambitieux mais raisonnable, n'est que légèrement manqué (0,476), ce qui situe mes résultats dans un ordre de grandeur crédible par rapport à l'état de l'art.

1	2	3	4
Décision prévisionnelle J'ai abandonné le fine-tuning QLoRA au profit du RAG pur — contraintes de temps, de GPU et de reproductibilité. Choix assumé et documenté.	Qualité des données Vidéos non pré-étiquetées (40 % de familles manquantes), biais de langue EN $>$ FR. Traitements spécifiques appliqués à chaque limite.	Pertinence des features Confondant sur la durée (durée $\approx$ type de chaîne) ; petit volume $\rightarrow$ haute variance (GridSearch non concluant sur ce corpus).	Contraintes IT Infra entièrement gratuite $\rightarrow$ TF-IDF (pas d'embeddings denses), traitement 100 % CPU. Pas de GPU disponible pour le fine-tuning.

8. Bilan

Contribution scientifique et technique

J'ai livré une **chaîne complète et reproductible** reliant une problématique métier concrète à une solution opérationnelle. Deux modèles supervisés ont été construits et entraînés (régression pour le scoring viral + classification pour les familles), complétés par une couche de génération RAG. L'ensemble est exposé via une API FastAPI consommable par une application de production.

Ma contribution dépasse la simple modélisation : le projet inclut la définition d'une variable cible robuste et comparable entre niches, un pipeline d'ingestion étiqueté et reproductible, une analyse statistique rigoureuse de chaque choix de modélisation, et une interprétation SHAP des résultats pour en extraire des recommandations actionnables par le créateur.

Gain net et mesurable

0,00

Baseline triviale

DummyRegressor (Spearman)

0,36

Baseline raisonnable

Régression linéaire

0,48

Notre modèle

LightGBM — gain net mesurable

Insertion métier — Le « menu du jour »

L'API expose quotidiennement un **menu du jour** : 5 idées en français + 5 idées en anglais, classées par score viral prédit. Ce service est directement consommable par l'application de production de la chaîne.

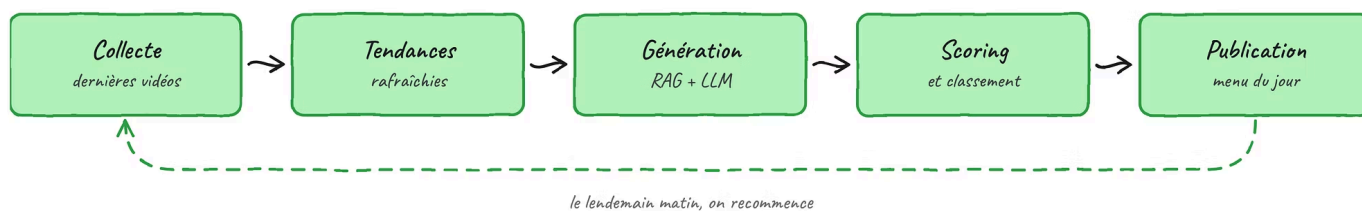


Schéma 2 — La boucle quotidienne. Inférence seule, aucun ré-entraînement : elle tourne sur processeur pour un coût quasi nul.

5 idées FR

Classées par `virality_score` prédit, ancrées sur les tendances et titres performants de la niche.

5 idées EN

Générées indépendamment pour le marché anglophone, avec les mêmes garanties de qualité.

Une seconde boucle, elle, n'est déclenchée qu'à la demande ou en cas de dérive détectée. C'est la seule qui touche aux modèles : elle les ré-entraîne, ré-indexe le corpus du générateur, compare le résultat au modèle en place et le versionne.

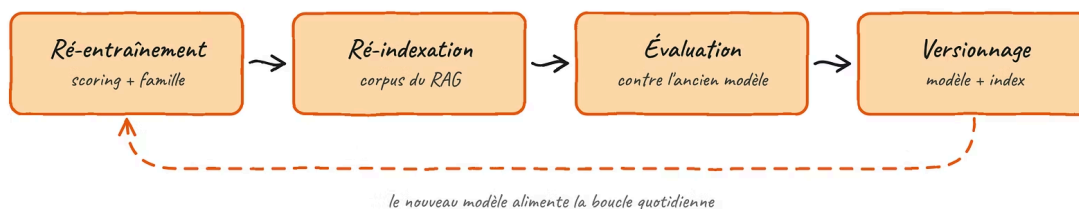


Schéma 3 — La boucle de mise à jour, seule étape qui ré-entraîne les modèles. Aucune carte graphique n'est nécessaire.

L'idéation devient ainsi un **service automatisé** : au lieu de chercher des idées à la main chaque matin, je reçois un classement argumenté et peux concentrer mon énergie sur la production et la qualité des vidéos à fort potentiel identifié.

- ✓ La littérature sur la prédiction de popularité avant publication rapporte des corrélations modérées similaires, ce qui situe ce résultat (Spearman = 0,476) dans un **ordre de grandeur crédible** par rapport à l'état de l'art.

9. Suite du projet et ouverture

Les résultats obtenus constituent une base solide pour plusieurs axes d'amélioration prioritaires. Ces pistes sont toutes fondées sur les limites que j'ai identifiées lors de l'exploration et de la modélisation, et visent à consolider la performance prédictive et la qualité de la génération.



Enrichissement des features

Intégrer les signaux de tendances Google Trends et les mots-clés déjà collectés mais pas encore exploités comme features du modèle de scoring. Ces signaux temporels pourraient capturer le « momentum » d'un sujet au moment de la publication.



Augmentation du volume

Étendre la collecte pour réduire la variance du modèle et permettre une optimisation hyperparamétrique plus robuste. Un corpus de 1 500 à 2 000 vidéos permettrait de tester le GridSearch dans des conditions plus favorables.



Embeddings sémantiques

Compléter les features de surface du titre par des représentations sémantiques denses (sentence-transformers, par exemple), capturant le sens du sujet au-delà des signaux syntaxiques (longueur, ponctuation, chiffres).



Comparaison de modèles

Tester CatBoost et XGBoost en comparaison directe avec LightGBM, et explorer l'ensembling (stacking) pour combiner leurs forces respectives sur ce type de données tabulaires de taille modeste.



Validation en conditions réelles

Je souhaite mesurer *a posteriori* la corrélation entre le score prédit par le modèle et la performance réelle des idées effectivement publiées sur la chaîne. C'est la validation ultime, en boucle fermée, de l'utilité du système.

10. Bibliographie

DataScientest — Supports de formation

DataScientest. Supports de cours et notebooks de formation Data Science & IA : Python, Machine Learning, MLOps. Source principale d'apprentissage utilisée tout au long du projet.

Stéphane Robert — Blog technique

Robert, S. Blog technique personnel couvrant le Machine Learning, le MLOps et le déploiement de modèles en production. Référence pratique pour la structuration du pipeline et les bonnes pratiques d'industrialisation. Lien : blog.stephane-robert.info

LightGBM — Documentation officielle

Ke, G. *et al.* (2017). *LightGBM : A Highly Efficient Gradient Boosting Decision Tree*. NeurIPS. Framework de gradient boosting utilisé pour le scoring viral et le classifieur de famille. Lien : lightgbm.readthedocs.io

FastAPI — Documentation officielle (FR)

Ramírez, S. *FastAPI : framework web moderne et rapide pour la création d'API avec Python*. Utilisé pour exposer le pipeline de génération et de scoring via une API REST. Lien : fastapi.tiangolo.com/fr

Streamlit — Documentation officielle

Streamlit Inc. Bibliothèque Python pour la création d'applications de données interactives. Utilisé pour la démonstration interactive du système. Lien : docs.streamlit.io

Google — API YouTube Data v3 (documentation FR)

Google. Documentation officielle de l'API YouTube Data v3. Source légale et officielle pour la collecte des métadonnées de vidéos publiques. Lien : developers.google.com/youtube/v3

11. Annexes

Description des modules de code

Module	Contenu
src/ingestion/	Collecte YouTube + Trends, étiquetage, mots-clés
src/features/	Variable cible, features anti-fuite, exploration
src/scoring/	Baselines, LightGBM, optimisation, SHAP
src/family/	Classifieur de famille (titre → famille)
src/generator/	Corpus RAG, retriever TF-IDF, génération, scoring
src/api/	API FastAPI — menu du jour 5 FR + 5 EN
src/mlops/	Tâche quotidienne, suivi de dérive
streamlit_app/, web/	Démonstration et interface de test

- 📁 L'ensemble du code est versionné et structuré pour une exécution reproductible de bout en bout, sans intervention manuelle, via un pipeline quotidien automatisé.

Technologies et environnement



Langage

Python 3.12



Analyse de données

pandas, numpy, scipy



Modélisation

scikit-learn, LightGBM ;
interprétabilité : SHAP



NLP / Génération

TF-IDF (scikit-learn),
modèle de langage via
API (RAG)



Collecte

YouTube Data API v3,
Google Trends
(pytrends)



Application

FastAPI (API) et Streamlit
(démonstration)



Industrialisation

Docker / Docker
Compose et GitHub
Actions (collecte
quotidienne
automatisée et
déploiement continu)



Infrastructure

Infrastructure gratuite,
CPU uniquement (aucun
entraînement sur GPU)